Side Activity 1: Introduction to SQLite

Database Systems Course

Learning Objectives

By the end of this tutorial, you will be able to:

- Install (or verify) the SQLite environment.
- Connect to an existing database file.
- Use SQLite meta-commands (dot-commands).
- Execute basic SQL `SELECT` queries to explore data.

Part 1: Setting Up the Environment

SQLite is a very lightweight relational database, but it is sufficient in most basic applications. Eventually, this course will encourage you to use the enterprise level relational database - Oracle. For now, SQLite will serve as a introduction to DBMSs, most of the contents learnt will transfer across naturally. Before we begin, we need to ensure you have the `sqlite3` tool and the sample data.

Step 1: Download the Data

1. Download the sample database `chinook.db` from the following link: `https://www.sqlitetutorial.net/sqlite-sample-database/` 2. Save the file in a folder that is easy to access (e.g., `Downloads` or create a folder named `db_dbExercises`).

Step 2: Open the Terminal

- **Windows:** Press `Win + R`, type `cmd`, and press Enter.
- **macOS:** Press `Cmd + Space`, type `Terminal`, and press Enter.
- **Linux:** Open your preferred terminal emulator.

Step 3: Navigate to Your Folder

Use the `cd` (change directory) command to go to where you saved `chinook.db`.

Example (if saved in Downloads)

```
cd Downloads
```

Part 2: Connecting to the Database

Now, let's start the SQLite command-line interface and open our database file.

Step 1: Launch SQLite

Run the following command in your terminal:

```
sqlite3 chinook.db
```

You should see a prompt that looks like this:

```
SQLite version 3.xx.x ...
Enter ".help" for usage hints.
sqlite>
```

Troubleshooting

Error: 'sqlite3' is not recognized?
This means SQLite is not in your system PATH. You may need to download the *sqlite-tools* zip file from `https://www.sqlite.org/download.html` and extract it to your folder, or ask a TA for help.

Part 3: SQLite Dot-Commands

SQLite has special commands that start with a dot (`.`). These are **not** SQL commands; they are instructions for the SQLite tool itself. They do not require a semicolon (`;`) at the end.

Task: Try the following commands

1. **List all tables** in the database:

   ```
   sqlite> .tables
   ```

   Question: How many tables do you see? Can you find the 'tracks' table?

2. **Show database information**:

   ```
   sqlite> .databases
   ```

   This shows the path of the currently attached database file.

3. **Show the schema**:

   Key Concept: What is a Schema?

   Think of the schema as the **blueprint** or design plan for your data. It defines the rules: what columns exist (e.g., "Title", "ArtistId") and what type of data they hold (e.g., text, numbers). It doesn't show the data itself, just the structure.

   ```
   sqlite> .schema albums
   ```

   This shows the `CREATE TABLE` statement used to build the table.

4. **Adjust display settings** (Optional but recommended): To make the output look like a nice table with headers:

```
sqlite> .mode column
sqlite> .headers on
```

Part 4: Basic SQL Queries

Now we will write standard SQL commands. **Important:** SQL commands must end with a semicolon (`;`).

Task 1: Peek at the Data

Key Concept: LIMIT vs. All Data

`SELECT *` grabs every single row. If a table has a million rows, this will crash your screen! `LIMIT n` tells the database to stop counting after n rows. It is perfect for a quick preview ("just show me the top 5").

Let's look at the first 5 rows of the `albums` table.

```
SELECT * FROM albums LIMIT 5;
```

Task 2: Selecting Specific Columns

Key Concept: Selecting Columns

Using `*` is convenient, but often you only need specific pieces of information. Listing column names (e.g., `Name`) is cleaner and more efficient than fetching unused data. Think of it as ordering specific dishes instead of the whole menu.

Instead of selecting everything (`*`), let's just get the names of artists.

```
SELECT Name FROM artists LIMIT 10;
```

Task 3: Basic Data Filtering

Key Concept: The WHERE Clause

While `LIMIT` cuts off the *quantity* of results, `WHERE` filters the *content*. It checks every row against your condition (like a price check) and only keeps the matches.

Find all tracks that cost more than $0.99.

```
SELECT Name, UnitPrice FROM tracks WHERE UnitPrice > 0.99;
```

Common Mistake: The "..." Prompt

If you press Enter and see `...>` instead of `sqlite>`, it means you forgot the semicolon (`;`).

- **Fix:** Type `;` and press Enter to finish the command.

Part 5: Self-Exercises (not graded)

Challenge 1: Find Specific Customers

The marketing team wants to send an email to all customers located in **Canada**.

- *Hint:* Look at the `customers` table. You need to filter by the `Country` column.
- *Goal:* Write a query to select the `FirstName`, `LastName`, and `Email` of all customers from Canada.

Challenge 2: Long Songs

We are looking for epic songs. Find all tracks that are longer than **5 minutes** (300,000 milliseconds).

- *Hint:* Check the `tracks` table. The column `Milliseconds` stores the duration.
- *Goal:* Select the `Name` and `Milliseconds` of these tracks.

Part 6: Exiting

When you are finished, you can exit the SQLite shell.

```
sqlite> .quit
```

End of Side Activity 1.